

A7 从自注意力到 Transformer

读完你会知道

先读 A5 和 A6。这一节把已经算过的“从其他位置取得线索”放进完整模型，沿一次文字生成解释为什么还要记录先后顺序、保留旧信息、整理数字大小。新的部件逐一说明，不要求先记名称。

读前回顾

A6 已手算过：每个词元对应一组数；用查询和键计算参考哪些位置，再按权重汇总值。这种计算叫自注意力。把它和其他数字处理步骤连成多层，得到一种常用的神经网络结构，叫 Transformer。它是结构名称，不是一个独立的人或聊天应用。这一节将具体走完这些步骤。

1 先看一张文字版结构图

大语言模型常用的 Transformer 可以先看成这样：

文字拆成词元 → 换成向量并加入位置信息 → 参考允许查看的其他位置 → 加入原有向量并整理数值 → 对每个位置再做一次独立计算 → 再加入原有向量并整理数值 → 重复多层 → 算候选词元的分数和概率 → 选一个接到原文后面。

这一节只讲一种用于继续生成文字的结构，不把所有使用 Transformer 的模型都说成完全相同。先看下面每个步骤解决什么问题；遇到模型设计的其他变化，可以在学完这条主线后再研究。

2 第一步：词元 变成能计算的数字

为什么第一步要查表？后面的注意力只能对向量做计算，而输入是文字。先把 词元 映射到数字向量，模型才有可以调整 and 组合的输入。

模型已有一张词表，以及词表中每个 词元 对应的一列数字。这个数字列表叫嵌入向量 (embedding)：词元是“鱼”时，模型就查到代表它的向量。向量不是小字典式的人工定义；数值在训练中调整，供后续计算使用。

可是，单靠 词元 本身，位置顺序不一定清楚。比如“狗咬人”和“人咬狗”包含相同词元，意思却不同。模型需要知道 词元 是第几个。我们把表示位置的向量加到每个 词元 的嵌入向量上：

$$h_i^{(0)} = e_i + p_i$$

e_i 是 词元 的嵌入， p_i 是位置 i 的表示， $h_i^{(0)}$ 是第一层看到的向量。位置表示可以按一组固定数学规则生成，也可以通过训练学习；不同模型设计不同。这里先理解“增加位置信息”的目的，不需背某一种位置公式。

小练习：如果两个词元的向量一模一样，但一个放在第 1 位、一个放在第 5 位，只加位置信息后会得到完全一样的输入吗？不会，因为两处加入的位置向量可能不同。这样模型有机会利用先后次序。

3 注意力如何让当前位置参考上下文

为什么还需要遮挡？训练文字里的后文已经写好，若当前位置能直接读到未来词元，预测题就会泄露答案；实际生成时未来文字也不存在。遮挡让训练中的每个位置只使用当时可见的内容。

第一层取得各位置的输入向量，用 A6 的三组矩阵分别相乘，得到查询 Q、键 K、值 V，再算点积、除以平方根、用 Softmax 求权重、按权重汇总值。这段计算会重新得到每个位置的向量，其中包含来自可查看位置的信息。

预测时有一个关键规则：在第 i 个位置预测下一个词元时，只能使用位置 i 及它前面的词元，不能偷看后面的文字。这种遮住未来位置的做法叫因果遮挡（causal mask）。它像考试时盖住未来题目答案；确保训练时和实际逐步生成时使用的信息一致。

以三个位置为例，可以看成分数表的形状如下；× 表示不允许查看：

当前查询位置	位置 1	位置 2	位置 3
1	可看	×	×
2	可看	可看	×
3	可看	可看	可看

计算时，禁止查看的位置不参加本行 Softmax 的分母求和，并直接赋予权重 0；允许位置之间重新分配总和为 1 的权重。软件常用负无穷这个特殊值实现遮挡，表示把该项彻底排除；仅设为很小的有限数，在数学上通常只能得到接近 0 的权重。生成第一个新词元时，模型参考提示内容；之后把新词元接上，再继续预测。

用 A6 的第 2 个位置验算：允许看位置 1、2，分数是 $[0, \sqrt{2}]$ ，位置 3 被遮住。用给定值 $e^0 = 1$ 、 $e^{\sqrt{2}} \approx 4.113$ ，前两项权重约为 $1 \div 5.113 = 0.1956$ 、 $4.113 \div 5.113 = 0.8044$ ，第三项为 0。这里必须对剩下的两项重新计算；不能直接把原三项权重的最后一项删掉，否则总和不再是 1。

这一步为何可以并行计算？训练阶段的正确答案文字已经给出，所以模型可一次构造许多位置的输入和遮挡表，平行算出各处的预测。遮挡会阻止它偷看右边答案。实际生成仍须逐个选出新词元，因为下一步输入依赖上一步选出的内容。

4 多个“阅读角度”：多头注意力

为什么一套注意力还不够？同一段文字里，位置之间可能同时存在不同的联系。一组参数只能给出一种变换后的匹配结果；多组参数让模型可以并行产生不同的线索组合，再交给后续计算。

一句话里可能同时有好几种相关关系：代词要联系前文的人或物，动词要联系执行动作的对象，转折词要联系前一句。只用一组查询和键，等于用一种匹配方式查看所有位置。

使用几套不同的查询、键、值矩阵，分别完成 A6 的计算，再合并结果，这叫多头注意力。一套计算叫一个头。沿用每个位置占一列的写法，要把同一位置各个头的输出上下接起来；例如 $[1,2]^T$ 和 $[3,4]^T$ 接成 $[1,2,3,4]^T$ 。再乘一个矩阵，把 4 个数变回后续步骤需要的数量。

下式中的 Attention 表示 A6 已完成的那套注意力计算；head 表示一个头的输出；r 是头的编号，上标 (r) 表示取第 r 套参数而不是乘方；MultiHead 表示把所有头合并后的结果。只是给已解释的步骤起英文简称。

$$\text{head}_r = \text{Attention}(\mathbf{WQ}^{(r)}\mathbf{X}, \mathbf{WK}^{(r)}\mathbf{X}, \mathbf{WV}^{(r)}\mathbf{X})$$

$$\text{MultiHead}(\mathbf{X}) = \mathbf{WO}[\text{head}_1; \text{head}_2; \dots; \text{head}_h]$$

方括号里的分号表示把向量上下拼接；h 是头的数量， $\mathbf{W}^\circ$ 是合并矩阵。可以把每个头想成一位手持不同问题清单的读者，读完后将笔记交给同一位整理者。**这只是帮助理解的比喻**：具体某个头学到什么关系，由数据和训练决定，不能预先给每个头分配“指代专员”等固定角色。

各个头处理同一段输入，但有各自的参数，所以可以得到不同结果。沿用这一节每列一个位置的写法，各头的结果矩阵上下拼接，再用合并矩阵变换。

5 每个位置再单独加工一次

为什么汇总了别处的信息后还要加工？注意力负责把相关位置的值带到当前向量，但这些值仍需要经过可学习的非线性变换。前馈网络在每个位置独立完成这一步。

注意力把其他位置的线索带过来，但每个位置还需要进一步加工自己现在持有的信息。Transformer 在注意力之后接一个小型前馈神经网络，**前馈**简单说就是输入沿固定方向经过几步计算、产生输出，不需要前馈网络之间互相查阅其他位置。

最简写法是：

FFN 是前馈网络的英文简称。x 是这个位置当前的向量；下面先用 $\mathbf{W}_1$ 、 $\mathbf{b}_1$ 进行乘加，再用 A4 的 ReLU 把负数变为 0，最后用 $\mathbf{W}_2$ 、 $\mathbf{b}_2$ 再进行乘加。下标 1、2 区分前后两组参数。

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

矩阵 $\mathbf{W}_1$ 、 $\mathbf{W}_2$ 和偏置 $\mathbf{b}_1$ 、 $\mathbf{b}_2$ 都是训练中调整的参数，ReLU 已在 A4 讲过。这段计算对每个位置单独执行，整段文字共用相同参数。这样注意力负责“把位置间的线索放在一起”，前馈网络负责“在一个位置上变换组合后的表示”。

6 保留旧信息并稳住数字

一层新计算时，若只把结果完全替换旧向量，可能丢掉有用信息，深层计算也会变得难学。因此每个部件旁有一条**残差连接**：先把部件的输出与原来的输入相加。

$$u = x + \text{Sublayer}(x)$$

这里 Sublayer 表示注意力或前馈网络中的某一个计算部件。相加要求两边形状相同；所以两个部件的输出都会保持与输入向量相同的长度。可把这理解为：“拿到新处理结果，同时把原有笔记也保留下来”。

反复相乘相加后，向量里数字的尺度可能变化很大。我们可以把每个位置的这一组数先减去本组平均值，再按本组数字的分散程度调整尺度；这种处理叫层归一化，英文 LayerNorm。它每次都根据当前这一组数计算，与 A3 用训练集确定固定输入缩放规则的做法不同。

先算 [1,3] 的例子：平均值是 $(1+3)\div 2 = 2$ ；每项减平均值得到 $[-1,1]$ 。把两项差分别平方后再平均，得到 $(1+1)\div 2 = 1$ ，这个数叫方差；方差的平方根叫标准差，这里也是 1。将 $[-1,1]$ 除以标准差，仍得 $[-1,1]$ 。这就完成了基本整理。

完整式子沿用这个步骤。 v_j 是第 j 个分量， d 是分量总数； μ 表示平均值，即全部分量之和除以 d ； σ^2 表示方差，即各项与平均值的差的平方再求平均。 γ_j 和 β_j 是训练可调整的数字，分别负责乘和加。 ϵ 是很小的正数，避免所有分量相同时分母变为 0。

$$\text{LayerNorm}(v)_j = \gamma_j (v_j - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta_j$$

例如暂取 $\gamma = 1$ 、 $\beta = 0$ ，并忽略很小的 ϵ ，刚才 [1,3] 的整理结果就是 $[-1,1]$ ；取 $\gamma = 2$ 、 $\beta = 0.5$ 则变为 $[-1.5, 2.5]$ 。真实计算保留 ϵ ，结果会有很小差异。这个操作不是把数字限制在 0 到 1，也不保证它们永远落在某个固定范围；目的是便于连续计算。不同模型也可能先整理再计算，这一节只演示一种明确顺序。

残差连接里的“保留”也要准确理解。例如旧向量 [1,2]，新计算结果 [3, -1]，相加得到 [4,1]。它给旧信息提供一条直接参与输出的途径，并不是额外存下一份可以原样取出的旧笔记。

7 一层如何不断重复

把所有部件串成一层，可以得到这个顺序：

输入表示 x

- 带因果遮挡的多头自注意力
- 与 x 相加（残差），层归一化
- 对每个位置做前馈网络
- 与上一步结果相加，层归一化
- 输出这一层的表示

模型把这一层重复许多次。每一层都会更新各位置的表示，后续层可以利用已经混入的线索继续变换。这里说“每个词元的表示越来越丰富”只是直观描述；计算上始终是一组向量经过矩阵运算与非线性函数。

8 最后一层如何变成下一个词元

为什么不能直接把最后一层向量当成文字？它仍是一列内部数字，读者看不懂，也没有对应到词表里的候选。输出权重先给每个候选打分，再由 Softmax 得到选择下一个词元所需的概率。

生成到当前位置时，模型取最后一个位置的向量，乘以一组输出权重，把它变成词表里每个词元的一个分数。Softmax 把分数变成候选概率，然后依据使用时的选择规则挑一个词元。这个步骤正是 A5 的“下一个词元预测”。新词元加到输入后，新一轮 Transformer 又计算更新的表示和概率。

每次先生成一小块，再把自己的输出接回输入继续生成，这种方式叫自回归生成。“自回归”在这里就是给这条循环起名字，不要求再学一个新的计算公式。

整个过程连在一起：

阶段	模型做什么	已学到的概念
输入	词元 查嵌入向量，加入位置信息	向量、输入特征
上下文	遮挡未来位置后计算自注意力	点积、矩阵乘法、Softmax
变换	多头注意力、前馈、残差和归一化	神经网络、激活函数
输出	把最后位置变成词表分数和概率	A5 的下一个词元预测
继续	选出词元，接回输入后重复	自回归生成

完整小例子：用户输入“小猫趴在窗台上”。模型先为提示文字建立向量和位置信息。每层根据遮挡规定，让已有位置互相交换线索并完成额外加工。输出端给出下一个词元的概率，如“睡”“晒”“看”等；假定这一轮选中“睡”，新的上下文就变成“小猫趴在窗台上睡”。系统再对新上下文重复整套预测，逐步生成回答。例句只是示意，模型真实概率取决于训练出的参数。

9 Transformer 和 ChatGPT 是什么关系

Transformer 是模型结构的设计，说明有哪些计算部件以及它们如何连接。LLM 是经过训练、能处理文字并预测词元的大型模型；许多此类模型以 Transformer 为主体。ChatGPT 一类产品则会把模型与对话提示、系统指令、可能使用的工具和应用逻辑组合起来。

所以“Transformer 会回答问题”是方便但不精确的说法：Transformer 提供处理与预测的计算结构；模型经训练后能按上下文产生输出；应用负责把用户请求交给模型并处理最终回答。三者相关，但不应当作同一个名词。

自己试一试

1. “猫在椅子旁，它打了个哈欠。”如果正在生成“它”之后的内容，模型能否偷看句子中尚未生成的未来词？为什么？
2. 为什么要把位置表示加到词元的嵌入向量中？给一个调换顺序会改变含义的例子。
3. 注意力已经混合不同位置的线索。前馈网络随后做什么？它会不会跨位置查询其他词元？
4. 如果一个注意力头输出 2 个数字、另一个输出 2 个数字，拼接后有几个数字？拼接后为什么还可再乘 W^O ？
5. 请用自己的话走一遍“用户输入一段文字，模型生成下一个词元”的流程，至少包括嵌入、位置信息、注意力和输出概率。

参考答案

1. 不能。生成位置只能使用提示内容以及已经生成的词元；遮挡未来信息让训练和逐步生成时的条件一致。
2. 位置向量提供先后次序。例：“狗咬人”和“人咬狗”有相同三个汉字，次序改变后意思改变。
3. 前馈网络对每一个位置的向量分别做一组相同的矩阵计算与 ReLU，不会跨位置互相查询；跨位置的读取由注意力承担。
4. 拼成一个 4 维向量。 W^O 可以变换拼接后的数字，把多组输出合并回模型需要的大小。
5. 示例：文字先拆成词元，每个词元查到嵌入向量并加上位置信息；多层 Transformer 用遮挡后的注意力读取前文线索、逐位置加工；输出端计算词表中各词元的分数并经 Softmax 得到概率；选出的词元接回输入后，模型继续预测。

一页记忆

Transformer 先把词元变成带位置信息的向量，再通过多层“自注意力 + 前馈网络”处理上下文。因果遮挡确保生成时不偷看未来；多头用几组不同参数并行汇总线索；残差保留旧信息；层归一化稳定数值。输出层把最后的表示转成下一个词元的概率。

延伸阅读

延伸阅读：Vaswani 等，Attention Is All You Need, 2017，第 3 节。原文：
<https://arxiv.org/abs/1706.03762>。不需要先读这篇论文。